

CROWDBEATS — STRIPE CONTENT PLATFORM COMPLIANCE IMPLEMENTATION

You are working on the existing Crowdbeats V2 cross-platform application.

This is NOT a redesign-only task.

This is a production architecture, compliance, security, moderation, Stripe Connect, Firebase, testing, and documentation implementation.

Crowdbeats is a music platform serving:

- Fans
- Solo Musicians / Artists
- Bands
- Sponsors
- Administrators

Crowdbeats allows fans to financially support or tip musicians and bands.

Crowdbeats may also support artist campaigns.

Crowdbeats is NOT intended to:

- sell adult or sexually explicit content
- facilitate pornography
- sell access to prohibited content
- facilitate intellectual-property infringement
- host pirated music
- facilitate extremist content
- operate a custom stored-value wallet
- store raw payment card data
- operate as an unlicensed money transmitter
- facilitate cryptocurrency or virtual-currency payments
- sell Crowdbeats credits/tokens unless separately approved in the future

Stripe has classified or is reviewing Crowdbeats as a content-creation/content-platform business.

The architecture must therefore be implemented so Crowdbeats can truthfully and demonstrably satisfy Stripe's content-platform due-diligence requirements.

DO NOT simply create documentation claiming controls exist.

Every compliance claim must map to:

1. actual application functionality
2. backend enforcement
3. database records
4. security rules
5. audit logging
6. admin controls
7. automated tests
8. documented operational procedures

Do not modify existing functionality unnecessarily.

Do not delete working application features.

Do not deploy to production.

Work against Firebase emulators, Stripe test mode, mocks, or isolated development resources.

PRIMARY OBJECTIVE

Create a complete:

CROWDBEATS STRIPE CONTENT PLATFORM COMPLIANCE LAYER

covering:

1. Stripe Connect
2. creator onboarding
3. unique creator profile URLs
4. creator monetization eligibility
5. prohibited-content enforcement
6. automated moderation
7. manual moderation
8. user reporting
9. copyright/IP complaints
10. takedowns
11. demonetization
12. payout holds
13. repeat-offender enforcement
14. account suspension
15. account termination
16. audit trails
17. Stripe webhook security
18. financial ledger integrity

19. refund/dispute handling
 20. privacy/security
 21. compliance dashboards
 22. evidence-generation for Stripe review
-

PHASE 0 — READ-ONLY ARCHITECTURE AUDIT

Before editing any files:

Inspect the existing repository.

Identify:

- mobile application framework
- web application framework
- admin web application
- shared packages
- Firebase configuration
- Firebase Authentication
- Firestore collections
- Firebase Storage
- Cloud Functions
- App Check
- Stripe integration
- Stripe Connect implementation
- webhook endpoints
- creator profiles
- musician profiles
- band profiles
- fan profiles
- sponsor profiles
- campaigns
- tipping
- payouts
- moderation
- reporting
- Terms of Service
- Privacy Policy
- Acceptable Use Policy
- admin authorization
- audit logs

Do not assume functionality exists because documentation references it.

Verify code.

Create:

docs/compliance/STRIPE_CONTENT_PLATFORM_CURRENT_STATE_AUDIT.md

Classify every requirement as:

IMPLEMENTED PARTIALLY IMPLEMENTED MISSING INSECURE NOT VERIFIED

For every item provide:

- relevant file
- function/component
- collection/schema
- test coverage
- security concerns
- recommended remediation

STOP destructive changes if architecture conflicts are discovered.

Continue with additive implementation wherever safely possible.

PHASE 1 — STRIPE CONNECT ARCHITECTURE

Crowdbeats must use Stripe Connect for creator payment flows.

Fans are customers.

Solo musicians and eligible bands are creators/connected accounts.

Crowdbeats is the platform.

Stripe remains the financial system of record.

Do NOT build a custom Crowdbeats custodial wallet.

Implement or verify:

- Stripe Customer creation where necessary
- Stripe Connected Account creation
- Connect onboarding
- account-link generation
- onboarding refresh flow
- onboarding return flow
- requirements_due handling

- charges_enabled status
- payouts_enabled status
- disabled_reason
- account_capability status
- bank payout readiness
- creator verification state

Store only safe Stripe identifiers such as:

stripeCustomerId stripeConnectedAccountId paymentIntentId chargeId transferId payoutId refundId

Never store:

- full PAN/card number
- CVC/CVV
- raw bank account numbers
- Stripe secret keys in Firestore
- unrestricted payment tokens

Use server-side Cloud Functions or equivalent trusted backend for Stripe secret operations.

Never expose Stripe secret keys client-side.

Use Firebase Secret Manager / Google Cloud Secret Manager or the project's existing secure secret mechanism.

PHASE 2 — UNIQUE CREATOR URLs

Stripe's questionnaire requires unique URLs for creators.

Implement canonical public URLs such as:

`https://crowdbeats.ai/artist/{creatorSlug}`

and

`https://crowdbeats.ai/band/{bandSlug}`

Requirements:

- unique slug
- immutable internal creator ID
- slug collision protection
- reserved-word protection
- URL normalization

- deleted-account handling
- suspended-account handling

Each creator URL must correspond to exactly one Crowdbeats creator identity.

Store:

creatorId creatorType creatorSlug canonicalProfileUrl

When Stripe requires creator website/profile information during Connect integration, supply the correct canonical creator URL through the Stripe integration.

Do not fabricate URLs.

Write integration tests proving the URL belongs to the creator being onboarded.

PHASE 3 — CREATOR MONETIZATION ELIGIBILITY

Create a server-enforced monetization eligibility model.

Example state:

NOT_ELIGIBLE PENDING_PROFILE PENDING_IDENTITY PENDING_POLICY_ACCEPTANCE PENDING_STRIPE
PENDING_MODERATION ACTIVE LIMITED DEMONETIZED PAYOUT_HOLD SUSPENDED TERMINATED

A creator cannot receive monetized payments merely because the frontend shows a tip button.

Backend enforcement must validate eligibility.

Create a centralized function such as:

assertCreatorMayMonetize()

It should validate:

- active account
- supported creator role
- accepted Terms
- accepted Acceptable Use Policy
- accepted creator monetization policy
- Stripe Connected Account exists
- Stripe charges enabled
- creator not suspended
- creator not demonetized

- no prohibited compliance state
- creator profile approved where required

Never rely exclusively on frontend validation.

PHASE 4 — ACCEPTABLE USE POLICY

Create or update Crowdbeats Acceptable Use Policy.

It must clearly prohibit monetization of:

Adult / Sexual Content

Prohibit:

- nudity intended for sexual gratification
- pornography
- explicit sexual acts
- sexual services
- paid adult live streams
- sexual solicitation
- sexually explicit paid content

Intellectual Property Violations

Prohibit:

- unauthorized copyrighted works
- pirated music
- leaked albums
- unauthorized recordings
- stolen artwork
- trademark infringement
- rights-of-publicity violations
- unauthorized copyrighted media

Violent Extremism / Hate

Prohibit:

- promotion of terrorism
- violent extremist organizations
- encouragement or glorification of unlawful violence
- targeted hate

- protected-class harassment
- violent extremist recruitment

Also prohibit:

- fraud
- scams
- deceptive fundraising
- impersonation
- illegal goods
- illegal services
- malware
- payment abuse
- harassment
- threats
- exploitation

Create:

legal/ACCEPTABLE_USE_POLICY.md

legal/CREATOR_MONETIZATION_POLICY.md

legal/CONTENT_MODERATION_POLICY.md

legal/COPYRIGHT_POLICY.md

legal/REPEAT_OFFENDER_POLICY.md

Do not state legal conclusions that require attorney approval.

Include a clear marker:

LEGAL REVIEW REQUIRED BEFORE PRODUCTION PUBLICATION

PHASE 5 — POLICY ACCEPTANCE

Require creators to explicitly accept:

Terms of Service

Privacy Policy

Acceptable Use Policy

Creator Monetization Policy

Store versioned acceptance records.

Example:

policyAcceptances/{acceptanceId}

Fields:

userId policyType policyVersion acceptedAt ipMetadataHash if legally appropriate userAgentMetadata if appropriate sourceApp platform createdAt

Do not store unnecessary personal information.

Policy acceptance must be server timestamped.

If a material monetization policy changes, support forced reacceptance before further monetized transactions.

PHASE 6 — CONTENT MODEL

Every creator-generated piece of content must have ownership and moderation metadata.

Content should include:

contentId ownerUserId creatorId creatorType contentType createdAt updatedAt moderationStatus monetizationStatus visibilityStatus moderationRiskScore moderationReasons lastModeratedAt manualReviewRequired

Possible moderationStatus values:

PENDING APPROVED FLAGGED RESTRICTED REMOVED APPEALED

Possible monetizationStatus:

ELIGIBLE REVIEW DEMONETIZED PROHIBITED

Never permanently erase required audit evidence when content is removed.

Public content may be hidden while preserving compliance records.

PHASE 7 — AUTOMATED CONTENT SAFETY PIPELINE

Build an extensible programmatic moderation pipeline.

This must be real functionality, not documentation.

Evaluate appropriate supported moderation technologies available to the project.

Use provider abstraction so Crowdbeats is not locked into a single classifier.

Create:

ModerationProvider ModerationResult ModerationDecision

The pipeline should be capable of reviewing:

- text
- profile descriptions
- campaign descriptions
- images
- uploaded media metadata
- links
- usernames where appropriate

Create automated checks for categories including:

- sexually explicit material
- hate
- violent extremism
- threats
- harassment
- scams/fraud
- copyright indicators
- prohibited URLs
- malicious links

Do not falsely claim AI detection is infallible.

Use a risk-based model.

Example:

LOW MEDIUM HIGH CRITICAL

Automated moderation actions may include:

ALLOW REVIEW DEMONETIZE HIDE BLOCK_UPLOAD ESCALATE

CRITICAL content must not become monetizable pending review.

PHASE 8 — HUMAN MODERATION

Create an Admin moderation queue.

Admins with appropriate permissions must be able to:

- view report
- view creator
- view content
- review automated moderation signals
- approve
- reject
- remove content
- restrict visibility
- demonetize content
- demonetize creator
- impose payout hold
- suspend account
- terminate account
- issue warning
- mark false positive
- escalate case
- record notes

Require reason codes.

Never permit silent administrative moderation actions.

Every action must create an audit log.

PHASE 9 — REPORT BUTTON

Implement reporting functionality across Crowdbeats.

Users must be able to report:

- creator profile

- band profile
- post
- campaign
- image
- video
- message where appropriate
- live activity where applicable

Report reasons should include:

SEXUAL_CONTENT COPYRIGHT TRADEMARK IMPERSONATION HATE VIOLENT_EXTREMISM THREATS
HARASSMENT SCAM FRAUD ILLEGAL_ACTIVITY SPAM OTHER

Create Firestore collection:

contentReports

Fields should include:

reportId reporterUserId targetType targetId creatorId reason description status createdAt updatedAt
assignedModeratorId resolution resolvedAt

Prevent report-spam abuse.

Do not expose reporter identity to the reported creator.

PHASE 10 — COPYRIGHT / RIGHTS-HOLDER COMPLAINTS

Provide an external copyright complaint mechanism even when the reporter does not have a Crowdbeats account.

Create a public form such as:

/legal/copyright-report

Support:

- claimant name
- rights-holder relationship
- copyrighted work description
- allegedly infringing Crowdbeats URL
- requested action

- good-faith attestation
- contact information
- electronic signature where appropriate

Protect this information from public exposure.

Route cases into a restricted administrative rights-management queue.

Create:

rightsHolderReports

Do not expose claimant contact data to unauthorized users.

Include procedures for:

- review
- takedown
- creator notice
- repeat offender tracking
- restoration/counter-notice workflow where legally appropriate

Mark legal workflow for attorney review before launch.

PHASE 11 — REPEAT OFFENDER SYSTEM

Implement repeat-violation tracking.

Do not simply count raw reports.

Count verified policy violations.

Create enforcement levels such as:

LEVEL_0_CLEAR LEVEL_1_WARNING LEVEL_2_MONETIZATION_RESTRICTED LEVEL_3_SUSPENDED
LEVEL_4_TERMINATED

Support serious violations that can immediately escalate to higher levels.

Track:

verifiedViolationCount violationTypes violationDates priorWarnings priorSuspensions
currentEnforcementLevel appealState

Create centralized logic such as:

`evaluateRepeatOffenderStatus()`

Every enforcement decision must be auditable.

PHASE 12 — DEMONETIZATION

Build monetization enforcement independently from visibility enforcement.

A creator/content item may sometimes remain visible while monetization is disabled.

Backend payment creation must check monetization state.

Examples:

`creator.status == ACTIVE`

AND

`creator.monetizationStatus == ACTIVE`

AND

`stripeAccount.chargesEnabled == true`

AND

`creator.complianceHold != true`

Fail closed.

A creator marked DEMONETIZED must not be able to bypass restrictions by manipulating a client.

PHASE 13 — PAYOUT HOLDS

Implement compliance payout hold state.

When legally and contractually permissible, support:

NONE REVIEW RISK DISPUTE POLICY LEGAL STRIPE_RESTRICTION

Crowdbeats must not pretend to hold money outside Stripe.

Payout handling must use Stripe-supported mechanisms.

A Crowdbeats UI may display status such as:

Available Pending Under Review Payout Restricted

but Stripe remains the financial system of record.

Do not create fake internal balances representing spendable stored value.

PHASE 14 — FAN PAYMENT SAFETY

All payments must be created server-side or through Stripe-approved client/server payment flows.

For tipping:

Fan → Crowdbeats → trusted backend → Stripe → eligible connected creator

Before creating payment:

validate:

fan creator creator eligibility tip amount currency payment context App Check rate limits idempotency
content/compliance status

Apply Stripe idempotency keys.

Prevent duplicate tips caused by retries.

PHASE 15 — STRIPE WEBHOOK SECURITY

Implement or verify Stripe webhook endpoint.

Verify Stripe signatures.

Never trust unsigned webhook requests.

Support idempotent processing.

Track Stripe events.

Handle relevant events such as:

payment success payment failure refund dispute connected account updates requirements changes payout updates account restrictions

Never log sensitive payment data.

Create:

stripeWebhookEvents

Store only required metadata such as:

stripeEventId eventType processingStatus receivedAt processedAt relatedObjectId errorCode

Avoid storing complete raw Stripe payloads unless explicitly justified and privacy-reviewed.

PHASE 16 — FIREBASE SECURITY

Verify Firestore rules.

Fans must not be able to:

- activate creator monetization
- approve content
- remove moderation strikes
- change Stripe account IDs
- change payout states
- alter compliance status
- modify admin audit logs

Creators must not be able to:

- self-verify
- self-approve moderation decisions
- remove violation history
- clear payout holds
- alter Stripe webhook data
- impersonate another creator

Admins must use server-verified roles/custom claims.

Critical actions should occur through privileged backend functions.

Enable or verify App Check for supported clients.

PHASE 17 — ADMIN RBAC

Create granular administrative permissions.

Examples:

MODERATION_READ MODERATION_WRITE CONTENT_REMOVE CONTENT_RESTORE DEMONETIZE_CREATOR
SUSPEND_CREATOR TERMINATE_CREATOR PAYOUT_HOLD PAYOUT_RELEASE VIEW_RIGHTS_REPORTS
MANAGE_RIGHTS_REPORTS VIEW_STRIPE_STATUS REFUND_PAYMENT VIEW_AUDIT_LOGS
EXPORT_COMPLIANCE_REPORT

Do not give every admin unrestricted permissions by default.

PHASE 18 — AUDIT LOGS

Create immutable-style application audit records for sensitive actions.

Examples:

content_removed content_restored creator_demonetized creator_remonetized payout_hold_applied
payout_hold_released account_suspended account_terminated copyright_report_resolved
stripe_account_restricted refund_issued policy_acceptance_changed

Store:

eventId eventType actorId actorRole targetType targetId reasonCode timestamp requestCorrelationId
previousState newState

Redact sensitive fields.

Users must not be allowed to edit audit logs.

PHASE 19 — APPEALS

Create moderation appeal flow.

Creators should be able to appeal eligible moderation decisions.

Appeals must not automatically restore monetization.

Create:

moderationAppeals

Fields:

appealId creatorId caseId reason status createdAt reviewedAt reviewerId resolution

PHASE 20 — PRIVACY

Apply data minimization.

Do not store unnecessary sensitive personal data.

Never expose:

- bank account details
- identity documents
- Stripe secrets
- claimant private contact information
- moderation security signals
- private admin notes

Review every Firestore collection for:

user access creator access admin access server-only fields

Produce privacy documentation.

PHASE 21 — STRIPE COMPLIANCE DASHBOARD

Create an Admin section:

Compliance & Trust

Include:

Stripe Connect

- connected creators

- incomplete onboarding
- charges disabled
- payouts disabled
- Stripe restriction reasons

Moderation

- pending reviews
- automated flags
- removed content
- demonetized content

Reports

- open reports
- rights-holder complaints
- copyright complaints
- escalated reports

Enforcement

- warnings
- demonetized creators
- suspended accounts
- terminated accounts
- repeat offenders

Payments

- failed payments
- disputes
- refunds
- payout restrictions

Do not display private Stripe financial information beyond what authorized administrators need.

PHASE 22 — COMPLIANCE EVIDENCE MODE

Create a way for authorized Crowdbeats administrators to generate an internal Stripe compliance evidence report.

It should answer:

1. Is Stripe Connect implemented?
2. How are connected creators onboarded?
3. Does every creator have a unique URL?

4. How is creator monetization eligibility enforced?
5. What prohibited content categories exist?
6. Where are policies published?
7. How is content automatically evaluated?
8. How is manual moderation performed?
9. How can users report content?
10. How can rights holders report content?
11. What happens to repeat offenders?
12. How are creators demonetized?
13. How are payouts restricted?
14. How are audit records preserved?
15. What test coverage proves these controls?

Generate:

docs/compliance/STRIPE_CONTENT_CREATOR_DUE_DILIGENCE_EVIDENCE.md

The report must distinguish:

IMPLEMENTED

from

PLANNED

Never label planned controls implemented.

PHASE 23 — STRIPE QUESTIONNAIRE CONTROL MATRIX

Create:

docs/compliance/STRIPE_QUESTIONNAIRE_CONTROL_MATRIX.md

Use columns:

Stripe Requirement Crowdbeats Answer Implementation Status Code Location Backend Enforcement UI
Location Policy Location Tests Evidence Remaining Risk

Map Stripe questionnaire requirements including:

- Stripe Connect
- unique creator URLs

- transaction categories
- account/login behavior
- prohibited content policy
- programmatic detection
- demonetization
- content removal
- reporting
- rights-holder complaints
- repeat violators
- formal attestation readiness

Do NOT recommend answering YES unless required implementation is genuinely operational.

PHASE 24 — AUTOMATED TESTS

Create tests covering:

1. creator without Stripe Connect cannot monetize
2. creator with incomplete onboarding cannot monetize
3. suspended creator cannot receive new tips
4. demonetized creator cannot receive new tips
5. prohibited content cannot become monetized
6. reported content enters moderation queue
7. copyright complaint enters rights-management queue
8. ordinary creator cannot access another creator's reports
9. creator cannot clear own violation
10. creator cannot alter Stripe account ID
11. fan cannot modify payout state
12. unsigned Stripe webhook rejected
13. duplicate Stripe webhook handled idempotently
14. duplicate payment requests handled idempotently

15. admin action produces audit log
16. repeat verified violations escalate enforcement
17. banned creator cannot create new monetized content
18. hidden content is no longer publicly discoverable when required
19. payout hold status blocks applicable payout initiation logic
20. policy reacceptance required following material policy update
21. unauthorized admin cannot perform moderation action
22. rights-holder contact information remains private
23. Firestore rules reject unauthorized compliance writes
24. Storage rules reject unauthorized media access
25. App Check protections operate on critical endpoints

Use Firebase Emulator Suite whenever possible.

Use Stripe test mode.

Never use production user data.

PHASE 25 — END-TO-END TEST

Create a deterministic end-to-end compliance scenario:

1. create fictitious musician
2. accept policies
3. begin Stripe Connect onboarding
4. complete test onboarding
5. create artist profile
6. verify canonical public URL
7. approve creator eligibility
8. fan discovers creator
9. fan sends Stripe test tip
10. payment succeeds
11. transaction appears in activity
12. creator content is flagged

13. user reports content
14. Admin reviews content
15. Admin verifies violation
16. content demonetized
17. creator receives enforcement strike
18. new tip attempt is blocked where creator monetization is suspended
19. enforcement audit event created
20. creator submits appeal
21. Admin resolves appeal

Document expected results.

PHASE 26 — DOCUMENTATION

Produce:

docs/compliance/STRIPE_CONTENT_PLATFORM_CURRENT_STATE_AUDIT.md

docs/compliance/STRIPE_CONTENT_CREATOR_DUE_DILIGENCE_EVIDENCE.md

docs/compliance/STRIPE_QUESTIONNAIRE_CONTROL_MATRIX.md

docs/compliance/STRIPE_CONNECT_ARCHITECTURE.md

docs/compliance/CONTENT_MODERATION_ARCHITECTURE.md

docs/compliance/COMPLIANCE_OPERATIONS_RUNBOOK.md

docs/compliance/STRIPE_PRODUCTION_READINESS_CHECKLIST.md

legal/ACCEPTABLE_USE_POLICY.md

legal/CREATOR_MONETIZATION_POLICY.md

legal/CONTENT_MODERATION_POLICY.md

legal/COPYRIGHT_POLICY.md

legal/REPEAT_OFFENDER_POLICY.md

PHASE 27 — PRODUCTION READINESS GATE

Do not approve production release until:

- ☐ Stripe Connect implemented
- ☐ creator URLs verified
- ☐ webhook signatures verified
- ☐ idempotency implemented
- ☐ creator eligibility server-enforced
- ☐ prohibited-content rules published
- ☐ moderation pipeline operational
- ☐ human moderation operational
- ☐ reporting operational
- ☐ copyright reporting operational
- ☐ repeat-offender enforcement operational
- ☐ demonetization operational
- ☐ payout restriction mechanism operational
- ☐ audit logs operational
- ☐ Firebase security rules tested
- ☐ App Check verified
- ☐ payment tests passing
- ☐ moderation tests passing
- ☐ E2E compliance test passing
- ☐ Stripe questionnaire control matrix contains no unsupported YES attestations

[] legal policies reviewed before production publication

[] current Stripe Restricted Businesses requirements manually reviewed before launch

CRITICAL ARCHITECTURAL RULE

Crowdbeats must NEVER create a fake internal wallet where Crowdbeats itself stores user money as spendable stored value.

The Crowdbeats UI may show Stripe-backed:

Pending Balance

Available Balance

Payout History

Bank Account ••••1234

but Stripe must remain the financial system of record.

Payouts to musicians/bands must occur through supported Stripe Connect mechanisms.

CRITICAL CONTENT RULE

Do not rely on the creator merely checking:

"I agree."

Technical enforcement must exist.

Backend code must prevent monetization when compliance conditions are not met.

CRITICAL DEVELOPMENT RULE

Do not claim something is implemented without code and tests proving it.

Use labels:

IMPLEMENTED PARTIAL PLANNED BLOCKED

Be conservative.

Stripe's questionnaire is a formal business attestation.

EXECUTION ORDER

Execute in this order:

1. Current-state audit
2. Architecture findings
3. Missing-control matrix
4. Stripe Connect implementation
5. creator monetization eligibility
6. creator URLs
7. policies
8. policy acceptance
9. moderation pipeline
10. reporting
11. rights-holder reporting
12. repeat-offender enforcement
13. demonetization
14. payout restrictions
15. admin compliance tools
16. audit logging
17. Firebase security
18. Stripe webhook hardening
19. tests
20. E2E test
21. evidence documentation
22. production readiness report

Do not skip the audit.

At the end, provide me with:

- A. files created
- B. files modified
- C. database collections added
- D. Cloud Functions added
- E. Firebase rules changed

F. Stripe integrations created

G. tests created

H. controls fully implemented

I. controls still incomplete

J. Stripe questionnaire questions that can truthfully be answered YES today

K. Stripe questionnaire questions that must remain NO until additional work is completed

L. any item requiring legal review

Do not deploy anything until I explicitly authorize deployment.